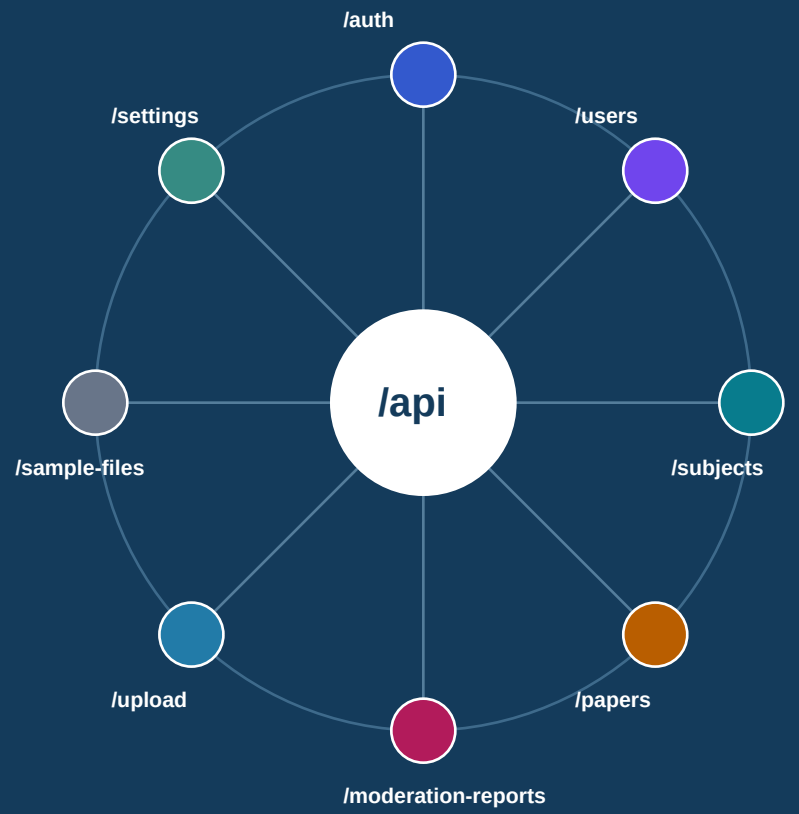
API REFERENCE

API Map

Examination Paper Moderation System

Wayamba University of Sri Lanka

24	13	8	5
operations	route files	resource groups	database models

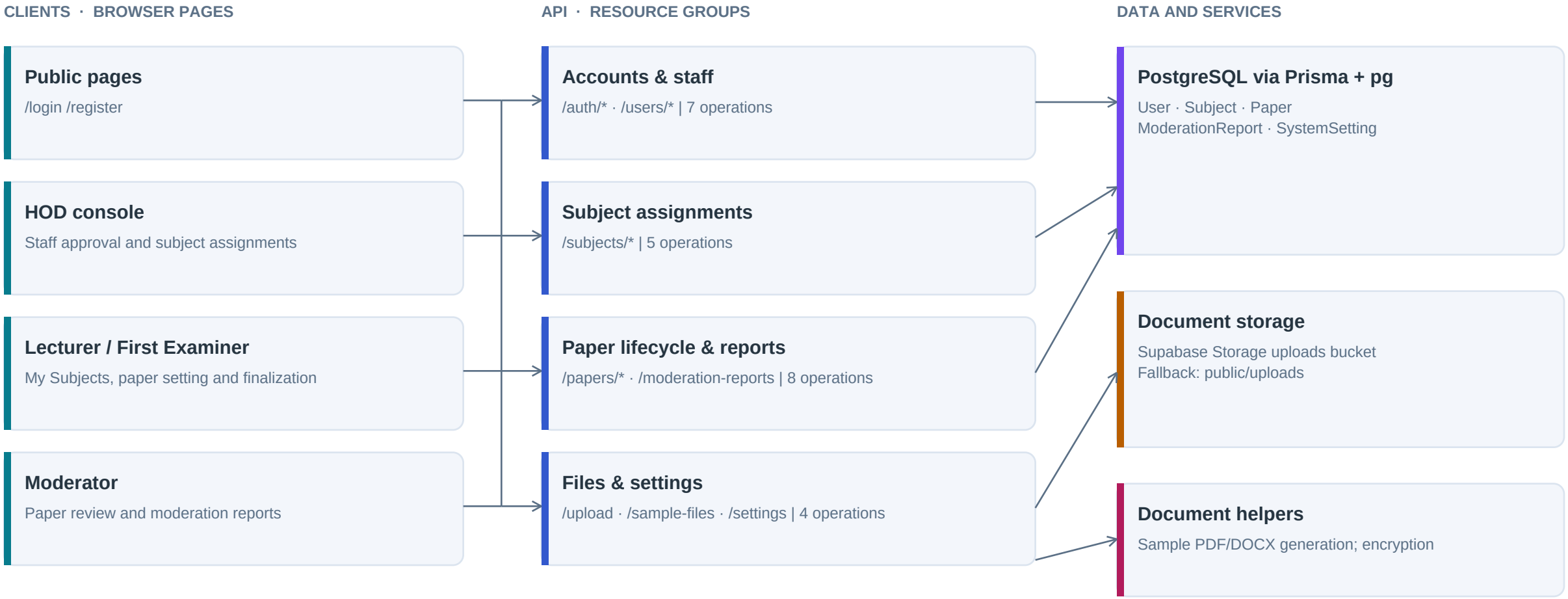


Source: dasikav-commits/Paper-Moderation-System · application code c55687a (Dasika lineage)

OVERVIEW

Architecture map

Browser workflows, API resource groups, database models and document services



Connections summarize typical usage, not access enforcement. Several handlers lack authenticated server identity; see Known limitations.

OVERVIEW

Endpoint index

Every operation and its intended UI consumer (1 of 2)

Method	Endpoint	What it does	Used by
POST	/api/auth/login	Validate credentials and selected department	Login page
POST	/api/auth/register	Create a lecturer account awaiting approval	Registration page
GET	/api/users	List staff, optionally by department	HOD / subject assignment views
POST	/api/users	Create a staff account	HOD staff management
PUT	/api/users/{id}	Update profile, role or Faculty Board approval	HOD / profile settings
PATCH	/api/users/{id}	Change account password	Profile settings
DELETE	/api/users/{id}	Delete staff and related records	HOD staff management
GET	/api/subjects	List subjects, staff and papers	HOD / subject views
POST	/api/subjects	Create subject and examiner assignments	HOD subject management
GET	/api/subjects/{id}	Load one subject and its assignments	Subject detail view
PUT	/api/subjects/{id}	Update subject or examiner assignments	HOD subject management
DELETE	/api/subjects/{id}	Delete subject, papers and reports	HOD subject management

OVERVIEW

Endpoint index

Every operation and its intended UI consumer (2 of 2)

Method	Endpoint	What it does	Used by
GET	/api/papers	List papers and virtual subject drafts	Lecturer / Moderator / HOD views
POST	/api/papers	Create an examination paper record	First Examiner
GET	/api/papers/{id}	Read a paper or virtual draft	Paper detail and review views
PUT	/api/papers/{id}	Update files, submit, revise or finalize	First Examiner workflow
GET	/api/moderation-reports	Read one report or list reports	Moderator / Lecturer / HOD views
POST	/api/moderation-reports	Create or update a report; request revision	Moderator report workflow
GET	/api/papers/{id}/download	Read protection password and file availability	Authorized download dialog
POST	/api/papers/{id}/download	Return encrypted or unlocked document	HOD / Moderator / First Examiner
POST	/api/upload	Store PDF, Word or signature image	Paper and report forms
GET	/api/sample-files	Generate example PDF or DOCX	Seeded/sample document links
GET	/api/settings	Read academic year and semester	Academic context / settings views
PUT	/api/settings	Persist setting key/value pairs	Settings workflow

WORKFLOW

Paper lifecycle

Paper.status values and the API calls used by the moderation workflow

Typical workflow only: the API does not enforce a complete state-transition graph. Revision can be resubmitted to UNDER_MODERATION.

#	Transition	Call	Guard and effect
1	Create -> DRAFT	POST /api/papers	Requires subjectId and lecturerId. Checks supplied lecturerId against assigned setter when present.
2	Submit / resubmit	PUT /api/papers/{id} {status: "under_moderation"}	Copies supplied document links to draft slots. First Examiner check only if actingUserId is supplied.
3	Review -> REVISION_REQUIRED	POST /api/moderation-reports	Upserts report by paperId. Changes state unless preserveStatus is truthy.
4	Finalize -> FINALIZED	PUT /api/papers/{id} {status: "finalized", filePassword, actingUserId}	Preserves original draft links; sets sentToHod and sentToHodAt. Password is only saved if supplied.

Client status strings: draft, under_moderation, revision_required, finalized. Alias submitted maps to UNDER_MODERATION; unknown values map to DRAFT. No immutable-finalization lock is present.

REFERENCE

Authentication and accounts

/api/auth · 2 operations

Method	Endpoint	Request	Success response	Rules and errors
POST	/api/auth/login Sign in	{email, password, department}	200 mapped user: id, name, email, role flags, approval, department, contact	400 missing fields. 401 invalid credentials or department. Plaintext password comparison. No token or session cookie issued.
POST	/api/auth/register Self-service staff sign-up	{name, email, password, department, confirmPassword?, contact?}	201 mapped user; lecturer role; facultyBoardApproved: false	400 required fields, invalid email, password <6, confirmation mismatch, non-10-digit contact or duplicate email. Email format checked; no university-domain allowlist.

Identity model

AuthContext stores user and selected role in localStorage.
Browser state is not a server-authenticated session.

New staff approval

Self-registered accounts start unapproved.
HOD workflow updates facultyBoardApproved through PUT /api/users/{id}.

REFERENCE

Staff administration & profile

/api/users · /api/users/{id} · 5 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	/api/users	query department?	200 raw user array	Sorted by name; department comparison normalized. Password fields are not excluded.
POST	/api/users	{name, email, password, department?, isDepartmentHead?, contact?, facultyBoardApproved?}	201 raw created user	400 required fields, invalid email, password <6, invalid contact, duplicate email. Approval defaults true; role selected by isDepartmentHead.
PUT	/api/users/{id}	{name?, email?, role?, department?, contact?, facultyBoardApproved?}	200 raw updated user	Updates provided fields. No server role check; limited explicit validation. Unknown record typically falls to 500.
PATCH	/api/users/{id}	{currentPassword, newPassword}	200 {success: true}	404 user missing; 400 incorrect current password. New password has no explicit strength check in this handler.
DELETE	/api/users/{id}	path id	200 {success: true}	Deletes reports and owned papers; clears subject links and removes ID from lecturerIds. Multiple steps, not wrapped in one transaction.

REFERENCE

Subjects & examiner assignments

/api/subjects · /api/subjects/{id} · 5 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	/api/subjects	query department?	200 enriched subject array	Includes staff relations, lecturers and papers/reports; ordered by code.
POST	/api/subjects	{code, name, semester, academicYear?, department?, firstExaminerId?, lecturerId?, lecturerIds?, moderatorId?, assignedByUserId?}	201 created subject	400 missing required fields, duplicate code, same First Examiner/Moderator, unapproved assigned staff. 403 if supplied actor is not HOD.
GET	/api/subjects/{id}	path id	200 enriched subject	404 subject not found. Includes related staff, papers and report data.
PUT	/api/subjects/{id}	{code?, name?, academicYear?, semester?, firstExaminerId?, lecturerId?, lecturerIds?, moderatorId?, assignedByUserId?}	200 updated subject	404 missing subject; 400 same examiner/moderator or unapproved selected staff. 403 supplied actor not HOD. Syncs draft paper owner when First Examiner changes.
DELETE	/api/subjects/{id}	path id	200 {success: true}	Deletes associated reports and papers before subject. No status lock or server HOD check.

HOD checks are conditional on assignedByUserId being supplied. Staff approval checks do not replace authenticated identity or ownership enforcement.

REFERENCE

Paper setting & finalization

/api/papers · /api/papers/{id} · 4 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	/api/papers	query lecturerId?, moderatorId?, department?	200 mapped paper array	Lecturer filter takes precedence over Moderator. Can append virtual drafts for assigned subjects without papers. No pagination.
POST	/api/papers	{subjectId, lecturerId, academicYear?, semester?, status?, paperUrl?, paperWordUrl?, markingSchemeUrl?, markingSchemeWordUrl?}	201 raw created paper	400 missing IDs. 404 subject. 403 supplied lecturerId differs from assigned setter when one exists. Saves draft and primary document links.
GET	/api/papers/{id}	path id; accepts virtual-{subjectId}	200 mapped paper or virtual draft	404 missing paper/subject. Maps status to lower case; returns document URLs, protection flag, report and HOD flags.
PUT	/api/papers/{id}	{status?, paperUrl?, paperWordUrl?, markingSchemeUrl?, markingSchemeWordUrl?, filePassword?, actingUserId?}	200 raw updated paper	404 missing paper. 403 setter mismatch only when actingUserId supplied and next state is UNDER_MODERATION or FINALIZED. No general transition lock.

Virtual IDs are read-time placeholders, not saved Paper records. Use POST /api/papers to create a real record before updating it, attaching a report or downloading protected files.

REFERENCE

Moderation reports

/api/moderation-reports · 2 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	/api/moderation-reports	query paperId?, moderatorId?, department?	200 single report with paperId; otherwise report array	paperId takes precedence and bypasses other filters. 404 if its report does not exist. Single report includes report file URLs; list includes paper summary.
POST	/api/moderation-reports	{paperId, moderatorId, remarks, additionalComments?, followUpActions?, moderatorSignatureUrl?, lecturerSignatureUrl?, reportPdfUrl?, reportWordUrl?, preserveStatus?}	201 raw upserted report	400 missing paperId/moderatorId/remarks. Upsert keyed by unique paperId. Sets paper to REVISION_REQUIRED unless preserveStatus is truthy. No assigned-Moderator identity check.

One report per paper

ModerationReport.paperId is unique.
Repeat submissions update the existing report rather than create a second one.

Report and document delivery

Report PDF/Word and signature fields store file URLs.
HOD views retrieve linked report information through the subject/report APIs.

REFERENCE

Downloads, uploads & settings

Document services and academic context · 6 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	/api/papers/{id}/download	query userId	200 {authorized, filePassword, hasPaperPdf, hasPaperWord, hasSchemePdf, hasSchemeWord}	403 access denied; 404 paper. Accepts department HOD, assigned Moderator or First Examiner by supplied user ID.
POST	/api/papers/{id}/download	{userId, password, fileType, mode?}	200 binary PDF/DOCX; no-store	401 wrong file password; 403 access; 404 paper. fileType: paperPdf, paperWord, schemePdf, schemeWord. mode unlocked returns raw bytes; otherwise encrypted.
POST	/api/upload	multipart: file; folder? (files default)	200 {url, path}	400 missing/disallowed file. PDF, DOC/DOCX, PNG/JPEG/GIF/WEBP. Supabase uploads bucket, else local public/uploads.
GET	/api/sample-files	query code?, kind?, format?, version?	200 generated PDF/DOCX	Defaults: CMIS 3134, paper, pdf, draft. kind scheme; format docx; version final select alternatives.
GET	/api/settings	none	200 {academicYear, semester}	Returns defaults if database read fails: 2025/2026 and 1.
PUT	/api/settings	JSON key/value object	200 {success:true}; warning on failure	Values stored as strings. Caught persistence failures still return success:true and warning.

RULES

State and permission matrix

Implemented checks, including conditional checks and missing locks

Call	Identity / role checks	State / business rules	Outcome
Create paper	Supplied lecturerId compared with assigned setter, if assigned	No lifecycle guard; supplied status accepted	403 mismatch; 404 subject
Update / submit / finalize	Setter check only if actingUserId supplied AND resulting state is UNDER_MODERATION or FINALIZED	No allowed-transition matrix; FINALIZED can be edited again	403 mismatch; 404 paper
Create / update subject assignments	Supplied assignedByUserId must be HOD; checked only if present	Reject same First Examiner/Moderator and unapproved selected staff	400 invalid assignment; 403 actor
Save moderation report	No authenticated assigned-Moderator check	Default sets REVISION_REQUIRED; preserveStatus bypasses state update	400 missing required data
GET / POST protected download	Supplied userId checked as department HOD, assigned Moderator or First Examiner	No FINALIZED-state requirement. POST also checks file password	403 access; 401 password; 404 paper
Delete user / subject	No authenticated HOD check	No paper status lock; deletes related records in separate operations	200 on success; exceptions 500
Read lists / change settings / upload	No general server-session check in reviewed handlers	No general role enforcement	Route-specific validation only

These are code-level observations, not a penetration-test result. Intended role separation in the UI is not equivalent to server-side API authorization.

DATA

Models and document shapes

Prisma schema · 5 models · PostgreSQL

Model	Important fields	Shape and relationships
User	id (cuid), email unique, name, password, role, contact, department, facultyBoardApproved	Roles: DEPARTMENT_HEAD, LECTURER, MODERATOR. Approval defaults true in schema; registration explicitly uses false.
Subject	id, code unique, name, academicYear, semester, department; firstExaminerId, lecturerId, moderatorId; lecturerIds JSON	Three optional User relations. lecturerIds is a JSON array, not a separate join model.
Paper	id, subjectId, lecturerId, academicYear, semester, status; primary and draft URL fields; filePassword, sentToHod, sentToHodAt	Belongs to Subject and User. Status: DRAFT, UNDER_MODERATION, REVISION_REQUIRED, FINALIZED.
ModerationReport	id, paperId unique, moderatorId, remarks JSON, additionalComments, followUpActions JSON, signature URLs, reportPdfUrl, reportWordUrl	At most one report per paper. Linked to moderator User. Report upload URLs and signature URLs are optional.
SystemSetting	id, key unique, value string, updatedAt	GET exposes academicYear/semester with defaults. PUT accepts key/value entries beyond those two keys.

Primary document slots: paperUrl, paperWordUrl, markingSchemeUrl, markingSchemeWordUrl.
Draft slots: draftPaperUrl, draftPaperWordUrl, draftMarkingSchemeUrl, draftMarkingSchemeWordUrl.
All five models have updatedAt; User, Subject, Paper and ModerationReport also have createdAt.

CONVENTIONS

Responses and errors

Transport, identifiers, status codes and fallback behavior

Convention	Current implementation
Transport	Same-origin HTTP under /api; no version prefix. JSON normally; multipart/form-data upload; binary document responses.
Identifiers	String cuid database IDs; subject code used by sample generator. virtual-{subjectId} is a read-only placeholder.
Lists and filters	Plain arrays; optional department / staff / paper filters depending on route. No pagination contract.
Error payload	Usually {error: message}; handlers often expose caught exception text in 500 responses.
Success payload	Mixed: mapped objects, raw Prisma objects, arrays, {success:true}, file metadata or binary bytes.
Caching and fallback	Protected download POST sets Cache-Control: no-store. Settings may return defaults/warnings with 200; missing document slots may generate samples.

Code	Meaning	Used for
200	OK	Reads, updates, deletes, uploads, binary downloads and settings fallbacks
201	Created	Register, create user/subject/paper; report upsert (even when updating)
400	Bad request	Missing fields, duplicate email/code, validation or assignment failure
401	Unauthorized	Invalid login / department or incorrect file password
403	Forbidden	Access denial or conditional role / setter checks
404	Not found	Explicit missing user, subject, paper or report cases
500	Server error	Caught exceptions; some unknown-record mutations fall through to this

NOTES

Known limitations and recommended next steps

Source review of route handlers and helpers; no live security assessment was performed

#	Finding	What the code does	Recommended next step
1	No authenticated server session	Login returns a user object; localStorage holds UI identity. Reviewed handlers do not validate a session/JWT.	Issue a secure server-verified session and require it for protected routes.
2	Caller-controlled identity	assignedByUserId and actingUserId checks are conditional. Download checks trust supplied userId, not authenticated identity.	Derive actor IDs from the session; require role, department and ownership checks.
3	Plaintext passwords and sensitive responses	Account passwords are compared as plaintext. Raw user/nested responses may expose passwords. GET download returns the file password.	Hash account passwords; select safe response fields; redesign file-secret distribution.
4	Source documents can bypass download gate	Uploads may return public URLs. Original file links and sample generator are separate from encrypted download. unlocked mode returns raw bytes.	Use private storage; authorize every retrieval; decide whether unlocked mode should exist.
5	Weak state and input boundaries	No complete paper transition matrix; finalized records remain editable. No uniform payload/file-size validation or rate limiting.	Centralize validation, lifecycle rules, file limits, allowed source URLs and throttling.
6	Failure handling and hardening	Settings can report success on persistence failure. Multi-step deletes are not atomic. Internal errors are echoed; pg TLS verification is disabled remotely.	Return accurate failures; use transactions; log details privately; verify database certificates.